



UNEB
UNIVERSIDADE DO
ESTADO DA BAHIA

Quinto Relatório de Montagem OTTO

**Nícolas da Silva Dórea
Guilherme Pires dos Santos Guimarães
Jezreel Victor Sales Rocha**

Salvador – Bahia

Outubro, 2026

Nícolas da Silva Dórea
Guilherme Pires dos Santos Guimarães
Jezreel Victor Sales Rocha

Quinto Relatório de Montagem OTTO

Relatório de montagem do robô OTTO, descrevendo o procedimento realizado, na UNEB em Salvador durante os meses de Setembro e Outubro, constituindo-se como trabalho pontuado na disciplina de Informática Aplicada no curso de Engenharia Civil.

Docente: Robson Marinho da Silva

Salvador – Bahia
Outubro, 2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. OBJETIVO.....	4
3. DESENVOLVIMENTO.....	4
3.1 MONTAGEM FÍSICA.....	4
3.2 MODELAGEM LÓGICA.....	5
3.3 PROGRAMAÇÃO AVANÇADA.....	5
4. CONCLUSÃO	6
5. REFERÊNCIAS.....	6

1. INTRODUÇÃO

Este documento consolida o registro de todas as etapas de desenvolvimento do robô bípede OTTO DIY, um projeto prático desenvolvido ao longo da disciplina de Informática Aplicada. O trabalho abrangeu desde a montagem mecânica do chassi impresso em 3D e integração dos componentes eletrônicos até a modelagem lógica do sistema utilizando Redes de Petri e a elaboração de rotinas de código. O projeto exigiu a superação de diversos desafios de hardware e software, culminando na criação de um dispositivo autônomo capaz de realizar leituras espaciais, movimentação coordenada e tomada de decisão lógica.

2. OBJETIVO

O objetivo principal deste projeto foi construir, integrar e programar o robô bípede OTTO DIY para atuar de forma autônoma e responsiva ao ambiente. De forma específica, buscou-se consolidar a montagem física superando as limitações estruturais e aplicar ferramentas avançadas, como a modelagem em Redes de Petri e o uso de Inteligência Artificial, para gerar os códigos de três comportamentos complexos definidos pela equipe: uma coreografia de dança, um sistema de trena digital para topografia básica e um algoritmo de desvio de obstáculos.

3. DESENVOLVIMENTO

3.1 MONTAGEM FÍSICA

A construção do robô iniciou-se pela fixação mecânica dos motores das pernas e da base, etapa que exigiu adaptações devido a divergências nos manuais fornecidos e infraestrutura de montagem. A integração do circuito foi realizada alocando o microcontrolador Arduino Nano no Shield de expansão e conectando os componentes de forma estratégica para evitar curtos e mau contato. As portas lógicas foram rigorosamente mapeadas: o sensor ultrassônico (HC-SR04) foi conectado nas portas 8-S (Trigger) e 9-S (Echo); os servomotores assumiram as portas GVS; e o Buzzer foi ligado aos pinos 13-S (positivo) e 13-G (negativo).

Houve percalços físicos, como a incompatibilidade de tamanho do botão liga/desliga que dificultou o fechamento da carcaça, e o travamento persistente do servomotor da perna esquerda, que exigiu a desmontagem completa e recalibração dos eixos a 90 graus para garantir a estabilidade do robô.

3.2 MODELAGEM LÓGICA

Antes da programação final, o comportamento do robô foi mapeado através de Redes de Petri do tipo Lugar/Transição. A metodologia PIM (*Platform Independent Model*) dividiu o sistema em três camadas: o Modelo da Estratégia (que dita o objetivo final da missão), o Modelo do Controle (que processa os dados dos sensores e envia comandos) e o Modelo do Robô (que executa fisicamente o movimento nos atuadores). Essa organização visual permitiu estruturar os algoritmos com clareza antes de inseri-los no Arduino IDE.

3.3 PROGRAMAÇÃO AVANÇADA

Após solucionar falhas iniciais de software (como portas não reconhecidas e erros em bibliotecas básicas), a equipe utilizou ferramentas de Inteligência Artificial para auxiliar na geração e estruturação do código C++ do robô, visando implementar com precisão três movimentos estratégicos:

- **Movimento 1 - Dança:** Foi gerado um algoritmo de movimento coreografado onde a instrução principal consiste no robô levantar a perna esquerda e flexionar o pé enquanto a mesma está levantada algumas vezes, validando a sincronia e o limite de torque dos motores em ações isoladas.
- **Movimento 2 - Trena (Aplicação em Engenharia Civil):** Desenvolveu-se um código focado em medição de distâncias. O robô atua como uma trena andante: ele marca o ponto de origem e caminha em linha reta contando os próprios passos padronizados. Ao detectar um objeto ou parede à sua frente através do sensor ultrassônico, os motores são interrompidos. Em seguida, o sistema calcula e sinaliza matematicamente a distância percorrida entre o ponto inicial marcado e o ponto de parada.
- **Movimento 3 - Corrida com Obstáculos:** Para garantir a integridade do robô durante a navegação livre, a IA ajudou a codificar um protocolo defensivo. O robô marcha para a frente avaliando o trajeto; contudo, ao encontrar um objeto abaixo da distância de segurança, ele não tenta girar imediatamente (o que poderia causar colisões). O código instrui o OTTO a primeiro ir para trás (recuar) e, somente após ganhar espaço seguro, ele rotaciona para desviar do obstáculo e retomar o percurso.

4. CONCLUSÃO

A jornada de montagem do OTTO DIY provou-se um excelente exercício prático de integração entre mecânica, eletrônica e software. Todas as adversidades, desde parafusos perdidos e motores travados até bugs no ambiente de compilação, foram resolvidas analiticamente. A adoção das Redes de Petri forneceu o rigor matemático necessário para entender o fluxo de dados, enquanto o uso de Inteligência Artificial para gerar o código final otimizou a implementação dos comportamentos de Dança, Trena Digital e Desvio de Obstáculos.

O sistema entregue cumpre todos os requisitos avaliativos da disciplina. Além disso, a capacidade de medição espacial (Trena) demonstra a viabilidade de aplicar conceitos de robótica na Engenharia Civil. Como projeção futura, sugere-se a adição de um sensor infravermelho no Shield para detectar focos de incêndio em canteiros de obras, complementado esteticamente por um capacete de engenheiro impresso em 3D, ampliando a utilidade do dispositivo.

5. REFERÊNCIAS

MARINHO, R. **Repositório Uni Robótica — Petri-nets: rede-blocos.hps**. GitHub, 2026. Disponível em: https://github.com/UniRobotica/Petri-nets/blob/main/Exemplos/Exemplos_HPSim/rede-blocos.hps. Acesso em: 4 out. 2026.

WEB HP SIM. **Web HP SIM Simulator**. [S. l.], 2026. Disponível em: <https://hpsim.vercel.app/>. Acesso em: 4 out. 2026.

PIPE2. **Platform Independent Petri Net Editor 2**. SourceForge, [S. d.]. Disponível em: <https://pipe2.sourceforge.net/>. Acesso em: 4 out. 2026.

UNIVERSITÄT HAMBURG. **Petri Nets tools database: HP Sim**. Hamburgo, [S. d.]. Disponível em: <https://www.informatik.uni-hamburg.de/TGI/PetriNets/tools/db/hpsim.html>. Acesso em: 4 out. 2026.